

LINGUAGEM DE PROGRAMAÇÃO C++

Curso de Capacitação em Desenvolvimento Full Stack - MÓDULO 08 (40H)

Elaborado por: Prof(a). Esp. Érika D. G. R. dos Santos

PROGRAMAÇÃO ORIENTADA A OBJETOS

AULA 9: LINGUAGEM DE PROGRAMAÇÃO C++

Programação Orientada a Objeto

Introdução a POO (Ontem)

- Classe: "molde" ou "projeto" (abstrato)
- Objeto: instância da classe (concreto)
- Atributos: características (dados)
- Métodos: comportamentos (funções)

Encapsulamento

SUMÁRIO

- 1 - Conceito Básico
- 2 - Atributos publicos/privados e controlado
- 3 - Métodos getter e setter
- 4 - Exercício de fixação

O que vamos aprender?

Encapsulamento

Aprender como o encapsulamento permite proteger os atributos de uma classe em C++, controlar seu acesso e garantir integridade.



Conceitos Básico

Encapsulamento

O que é Encapsulamento?

Encapsulamento é o princípio da POO que esconde os detalhes internos de uma classe, expondo apenas o necessário ao usuário.

Benefícios:

- Proteção de dados sensíveis
- Redução de erros acidentais
- Clareza na interface pública da classe
- Facilidade na manutenção do código

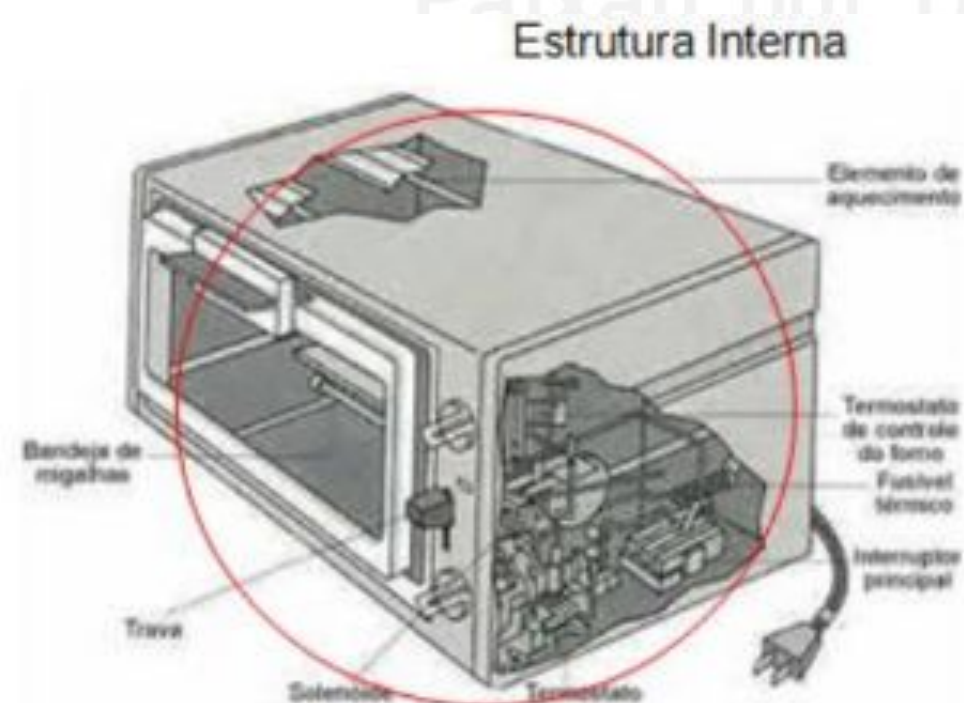


Conceitos Básico

Encapsulamento

Encapsulamento

Separação dos aspectos externos de um objeto, acessíveis por outros objetos, de seus detalhes internos de implementação, que ficam ocultos dos demais objetos.



Conceitos Básico

Encapsulamento

C++ possui três níveis de visibilidade para atributos e métodos:

Tipo	Sintaxe	Acesso
Público	public	Acessível de QUALQUER LUGAR do código. (Padrão para métodos)
Protegido	protected	Acessível dentro da classe e por classes filhas (herança).
Privado	private	Acessível APENAS dentro da própria classe. (Padrão para atributos)

Regra de Ouro:

Atributos devem ser private.

Métodos que interagem com o objeto devem ser public.

Exemplo: Classe sem Encapsulamento

```
#include <iostream>
using namespace std;

class ContaBancaria {
public: // Tudo é público - RUIM!
    string titular;
    double saldo;
};

int main() {
    ContaBancaria conta;
    conta.titular = "João";
    conta.saldo = 1000.0;

    // Problema: acesso direto e sem controle!
    conta.saldo = -500.0; // Saldo negativo permitido!
    cout << "Saldo: " << conta.saldo << endl;

    return 0;
}
```

Problemas:

- ✗ Qualquer um pode modificar o saldo diretamente.
- ✗ Não há validação de dados (saldo negativo permitido).
- ✗ Quebra da integridade dos dados.

Exemplo: Classe com Encapsulamento

```
class ContaBancaria {  
private: // Atributos PRIVADOS  
    string titular;  
    double saldo;  
  
public: // Métodos PÚBLICOS  
    // Construtor  
    ContaBancaria(string t, double s) : titular(t), saldo(s) {}  
  
    // Getters (leitura controlada)  
    string getTitular() const { return titular; }  
    double getSaldo() const { return saldo; }  
  
    // Setters (escrita controlada com validação)  
    void depositar(double valor) {  
        if (valor > 0) {  
            saldo += valor;  
            cout << "Depósito de R$" << valor << " realizado." << endl;  
        } else {  
            cout << "Valor inválido para depósito!" << endl;  
        }  
    }  
}
```

```
    bool sacar(double valor) {  
        if (valor > 0 && valor <= saldo) {  
            saldo -= valor;  
            cout << "Saque de R$" << valor << " realizado." << endl;  
            return true;  
        } else {  
            cout << "Saldo insuficiente ou valor inválido!" << endl;  
            return false;  
        }  
    }  
};
```

Usando a Classe com Encapsulamento

```
ContaBancaria conta("João", 1000.0);

// Acesso somente através dos métodos públicos
cout << "Titular: " << conta.getTitular() << endl;
cout << "Saldo: R$" << conta.getSaldo() << endl;

// Tentativa de acesso direto (gera erro de compilação!)
// conta.saldo = -500.0; // ERRO: 'double ContaBancaria::saldo' is private

// Operações controladas
conta.depositar(500.0);
conta.sacar(200.0);
conta.sacar(2000.0); // Tentativa de saque inválido

cout << "Saldo final: R$" << conta.getSaldo() << endl;
```

Por que Encapsular?

Vantagens do Encapsulamento

Benefício

Explicação

Controle de Acesso

Você decide como os dados podem ser lidos/modificados.

Validação de Dados

Impede que valores inválidos sejam atribuídos.

Segurança

Protege dados sensíveis contra alterações acidentais.

Flexibilidade

Pode mudar a implementação interna sem afetar quem usa a classe.

Manutenibilidade

Código mais organizado e fácil de manter.

Integridade

Garante que o objeto sempre esteja em um estado válido.

Getters e Setters

Getters e Setters: O que são?

Getters: Métodos que permitem **ler** o valor de um atributo privado.

Setters: Métodos que permitem **modificar** o valor de um atributo privado.

```
class Pessoa {  
private:  
    string nome;  
    int idade;  
  
public:  
    // Getter  
    string getNome() const {  
        return nome;  
    }  
  
    // Setter com validação  
    void setNome(string novoNome) {  
        if (!novoNome.empty()) {  
            nome = novoNome;  
        }  
    }  
};
```

```
    // Getter  
    int getIdade() const {  
        return idade;  
    }  
  
    // Setter com validação  
    void setIdade(int novaIdade) {  
        if (novaIdade > 0 && novaIdade < 150) {  
            idade = novaIdade;  
        } else {  
            cout << "Idade inválida!" << endl;  
        }  
    }  
};
```

Vamos praticar?



Ex. Fixação

Classe Aluno

Crie:

Atributos privados nome, nota

setNota() deve permitir apenas notas entre 0 e 10.
getNota() e getNome() para leitura.

Exemplo de uso:

```
Aluno a("Carlos", 8.5);  
a.set_nota(12); // Deve mostrar mensagem de erro
```



Getters e Setters na Prática

```
class Aluno {  
private:  
    string nome;  
    double nota;  
  
public:  
    // Construtor  
    Aluno(string nome, double nota) {  
        this->nome = nome;  
        // Validação inicial da nota  
        if (nota >= 0 && nota <= 10) {  
            this->nota = nota;  
        } else {  
            this->nota = 0;  
            cout << "Nota inicial inválida! Definida como 0." << endl;  
        }  
    }  
};
```

```
    string get_nome() {  
        return nome;  
    }  
  
    // Getter da nota  
    double get_nota() {  
        return nota;  
    }  
  
    // Setter da nota com validação  
    void set_nota(double nova_nota) {  
        if (nova_nota >= 0 && nova_nota <= 10) {  
            nota = nova_nota;  
            cout << "Nota atualizada para " << nova_nota << endl;  
        } else {  
            cout << "Nota inválida! Digite um valor entre 0 e 10." << endl;  
        }  
    }  
};
```

Getters e Setters na Prática

```
void exibir_informacoes() {  
    cout << "Aluno: " << nome << " | Nota: " << nota << endl;  
}  
};  
  
int main() {  
    // Testando o código  
    Aluno a("Carlos", 8.5);  
    a.exibir_informacoes();  
  
    a.set_nota(12); // Deve mostrar erro  
    a.exibir_informacoes();  
  
    a.set_nota(9.5); // Deve atualizar  
    a.exibir_informacoes();  
  
    return 0;  
}
```

Saída esperada:

```
Aluno: Carlos | Nota: 8.5  
Nota inválida! Digite um valor entre 0 e 10.  
Aluno: Carlos | Nota: 8.5  
Nota atualizada para 9.5  
Aluno: Carlos | Nota: 9.5
```

Exercício de Fixação 2

Classe Produto

Crie:

nome, preco como privados.

Getter e Setter com validação no preço (não pode ser zero ou negativo).

Teste com dois produtos e exiba os dados.



```
#include <iostream>
#include <string>
#include <iomanip>

using namespace std;

class Produto {
private:
    string nome;
    double preco;

public:
    // Construtor
    Produto(string nome, double preco) {
        this->nome = nome;
        this->preco = preco;
    }

    string get_nome() {
        return nome;
    }

    // Getter do preço
    double get_preco() {
        return preco;
    }

    // Setter do preço com validação
    void set_preco(double novo_preco) {
        if (novo_preco > 0) {
            preco = novo_preco;
            cout << fixed << setprecision(2);
            cout << "Preço do produto '" << nome << "' atualizado para R$ " << novo_preco << endl;
        } else {
```

```
        cout << "Preço inválido! O valor deve ser maior que zero." << endl;
    }
}

// Método para exibir informações formatadas
void exibir_informacoes() {
    cout << fixed << setprecision(2);
    cout << "Produto: " << nome << " | Preço: R$ " << preco << endl;
}

};

int main() {
    // --- Teste do sistema ---

    Produto produto1("Notebook", 3500.00);
    Produto produto2("Mouse", 80.00);
```

```
produto1.exibir_informacoes();  
produto2.exibir_informacoes();  
  
// Testando validações  
produto1.set_preco(-1500); // inválido  
produto2.set_preco(100);   // válido  
  
// Exibindo novamente após atualização  
cout << "\nApós atualização de preços:" << endl;  
produto1.exibir_informacoes();  
produto2.exibir_informacoes();  
  
return 0;  
}
```

Saída esperada:

```
Produto: Notebook | Preço: R$ 3500.00  
Produto: Mouse | Preço: R$ 80.00  
Preço inválido! O valor deve ser maior que zero.  
Preço do produto 'Mouse' atualizado para R$ 100.00
```

```
Após atualização de preços:  
Produto: Notebook | Preço: R$ 3500.00  
Produto: Mouse | Preço: R$ 100.00
```

Exercício de Fixação 3

Classe Carro

Atributos privados:

marca, velocidade

Métodos:

acelerar(valor) – aumenta a velocidade

frear(valor) – reduz, mas nunca abaixo de zero

Getters para exibir as informações



Getters e Setters na Prática

Exercício 1

Um restaurante deseja informatizar parte do seu atendimento. Para isso, você deve criar uma classe Restaurante que represente um estabelecimento, aplicando encapsulamento para proteger seus dados.

Getters e Setters na Prática

Classe Restaurante - Atributos privados:

- nome → nome do restaurante
- cardapio → lista de pratos disponíveis

Métodos:

- getNome() → retorna o nome do restaurante.
- setNome(novo_nome) → altera o nome apenas se for uma string não vazia.
- adicionarPrato(prato) → adiciona um novo prato ao cardápio somente se ainda não existir.
- listarCardapio() → exibe todos os pratos disponíveis.
- atenderCliente(nome_cliente, prato) → verifica se o prato está no cardápio.
- Se estiver → exibe mensagem confirmando o pedido.
- Se não estiver → exibe mensagem informando que o prato não existe.

Desafio Extra: Crie um restaurante chamado “Sabor Caseiro”. Adicione 3 pratos diferentes ao cardápio. Simule 2 clientes fazendo pedidos, sendo um deles de um prato inexistente. Altere o nome do restaurante usando o método setNome() e exiba novamente o nome com getNome().

Getters e Setters na Prática

Exercício 2

Imagine que você está desenvolvendo um sistema simples para gerenciar uma biblioteca.

A cada novo livro adquirido, ele deve ser registrado, e a biblioteca precisa manter uma lista atualizada de todos os títulos disponíveis.

Implemente o sistema aplicando o conceito de encapsulamento para proteger os dados da classe.

Getters e Setters na Prática

Classe: Biblioteca - Atributos privados:

nome → nome da biblioteca

livros → lista de livros disponíveis (inicialmente vazia)

Métodos:

getNome() → retorna o nome da biblioteca.

setNome(novo_nome) → altera o nome apenas se for uma string não vazia.

adicionarLivro(titulo) → adiciona um livro à lista somente se o título for válido (não vazio) e ainda não existir na lista.

listarLivros() → exibe todos os livros cadastrados.

Caso não haja livros, exibir a mensagem: "**Nenhum livro cadastrado.**"

Desafio Extra

Crie uma biblioteca chamada "Biblioteca Central". Adicione 3 livros diferentes. Tente adicionar um livro repetido (o sistema deve impedir). Exiba a lista de livros. Altere o nome da biblioteca usando o método setNome() e exiba o novo nome usando getNome().

Getters e Setters na Prática

Exercício 3

Uma escola precisa organizar os alunos matriculados.

Cada aluno possui informações pessoais que devem ser protegidas, e a escola deve controlar de forma segura a lista de alunos.

Implemente o sistema aplicando encapsulamento para proteger e validar os dados das classes.

Getters e Setters na Prática

Classe: Aluno - Atributos privados:

nome

idade

curso

Métodos:

getNome() → retorna o nome do aluno.

setNome(novo_nome) → altera o nome apenas se não for vazio.

getIdade() → retorna a idade.

setIdade(nova_idade) → altera apenas se a idade for um número maior que 0.

getCurso() → retorna o curso.

setCurso(novo_curso) → altera apenas se não for vazio.

apresentar() → retorna uma string apresentando o aluno:

"Nome: João | Idade: 20 | Curso: Informática"

Getters e Setters na Prática

Classe: Escola - Atributos privados:

nome → nome da escola

alunos → lista de alunos (inicialmente vazia)

Métodos:

getNome() / setNome() → controle de leitura e alteração do nome.

adicionarAluno(aluno) → adiciona um aluno à lista somente se for um objeto da classe Aluno.

listarAlunos() → exibe todos os alunos matriculados chamando o método apresentar() de cada um.

Caso não haja alunos, exibir a mensagem: "Nenhum aluno matriculado."

Desafio Extra

Crie uma escola chamada "Escola Futuro".

Crie 2 alunos com nome, idade e curso.

Adicione os alunos à escola.

Exiba a lista de alunos matriculados.

Teste alterações de atributos usando os métodos set.

DICA

Encapsulamento é como colocar os dados em um cofre: ninguém pode mexer neles sem a chave certa.



Parabéns

Encapsulamento é como colocar os dados em um cofre: ninguém pode mexer neles sem a chave certa.

Você aprendeu:

A diferença entre público, protegido e privado

Como proteger atributos com getter e setter

Como validar e controlar acesso a dados

"Dominar o encapsulamento é como ter um escudo no seu código.

Pratique, teste e proteja seus dados".

